AltaLex API 接入技术规范

1. 范围与约定

2. Key ID 生成（由客户生成）

3. Ed25519 密钥对生成（由客户生成）

4. HTTP 鉴权 Header

5. 签名原文（canonical string）

6. 多语言签名与请求示例

   6.1 Python

   6.2 Java

   6.3 Go

   6.4 Node.js

7. cURL（联调）

8. 调试清单（签名失败）

9. 密钥轮换与安全

附录 A 公钥与配置登记渠道

   A.1 当前渠道（公钥登记）

   A.2 其他配置（IP 白名单等）

附录 B 邮件模板（公钥注册与接入确认）

   B.1 客户 → AltaLex：公钥注册申请

   B.2 AltaLex → 客户：接入确认（不含私钥）

附录 C 可选 IP 白名单：邮件往来模板

   C.1 客户 → AltaLex：申请启用绑定

   C.2 AltaLex → 客户：已启用回函

10. 修订记录

项目	说明
文档版本	1.0

修订日期	2026-04-20
适用对象	客户侧研发、运维（集成 AltaLex HTTP API）

---

1. 范围与约定

本规范说明客户在**自有环境**生成 **Key ID** 与 **Ed25519** 密钥对、向 AltaLex **登记公钥**（**渠道与邮件模板见附录 A、附录 B**），以及使用**请求签名**访问 AltaLex API 的技术细节。

AltaLex 不生成、不接收、不存储客户私钥。 私钥材料仅存在于客户侧（密钥库、环境变量、HSM 等）。

Base URL（示例）： `https://api.example.com/v1`
具体路径以 AltaLex 发布的 OpenAPI / 路径清单为准；本规范仅规定**鉴权与签名**。

---

2. Key ID 生成（由客户生成）

- **归属**：Key ID 由客户在本地生成，与公钥一并提交 AltaLex 登记；须保证与 AltaLex 已登记主键**不冲突**（冲突时登记失败，需更换后重提）。
- **建议格式**： `pk_live_` + 高熵随机串（如 URL-safe Base64 子集），便于识别环境。
- **长度**：勿超过 AltaLex 告知的库表上限（常见为 **64** 字符）；过长将被拒绝。
- **熵**：随机部分应使用密码学安全随机数生成器（CSPRNG）。

Python 示例（生成 Key ID；实际栈请按需移植）：

```python
import secrets

key_id = "pk_live_" + secrets.token_urlsafe(22).rstrip("=")
# 自行校验总长度 ≤ 64；若超长则缩短随机段或改用更短前缀
```

---

3. Ed25519 密钥对生成（由客户生成）

在客户服务器或安全终端执行；**仅将Base64后的公钥（public_key_b64）**提交 AltaLex。**切勿**通过邮件、IM、工单发送私钥。

Python（PyNaCl）示例：

```python
import base64
import secrets
from nacl.signing import SigningKey

signing_key = SigningKey.generate() # 私钥
verify_key = signing_key.verify_key # 公钥

# 发往 AltaLex 登记的一行 Base64（verify key）
public_key_b64 = base64.b64encode(verify_key.encode()).decode()

# 私钥材料仅保存在贵司侧：常见为保存 signing_key.encode() 的 Base64，
# 或按所用库要求保存 32 字节 seed；加载方式见该库文档
```

公钥邮件正文模板见**附录 B.1**。

---

4. HTTP 鉴权 Header

每个业务请求须在 Header 中携带（名称大小写敏感，以实际网关要求为准）：

Header	说明
`X-API-Key-ID`	客户生成并已登记的 Key ID
`X-Request-Timestamp`	Unix 时间戳，**秒**，字符串
`X-Request-Nonce`	**16** 个十六进制字符（即 8 字节随机数的 hex）
`X-Ed25519-Signature`	对下文「签名原文」的 Ed25519 签名，**Base64** 编码
`Content-Type`	JSON 请求时使用 `application/json`

示例：

```
X-API-Key-ID: pk_live_L84_UR4VaR9z8OeqNPVqOq3nbeN2Jf2
X-Request-Timestamp: 1717323152
X-Request-Nonce: 9c76d9e45123faaf
X-Ed25519-Signature: X1psD/A1cy7S6qM/UK8liSUepB4L4+2URy+K35n9dc/sTXslr8DaoSp/2Thu43pZzbiwTcXTagRAQLAFa9tgCg==
Content-Type: application/json
```

时钟偏移：服务端通常允许 ±5 分钟；超出则返回时间相关错误（具体 HTTP 状态与业务码以网关为准）。

---

5. 签名原文（canonical string）

按以下顺序用 **ASCII 竖线** `|` 拼接为单行字符串（**无**首尾空格、**无**多余分隔符）：

```
{timestamp}|{nonce}|{HTTP方法}|{请求路径}|{查询字符串}|{Body的SHA256十六进制}
```

段	规则
`timestamp`	与 `X-Request-Timestamp` 一致
`nonce`	与 `X-Request-Nonce` 一致
`HTTP方法`	大写：`GET`、`POST`、`PUT` 等
`请求路径`	仅 path，**不含** host、**不含** 查询串；例如 `/contracts/list`
`查询字符串`	原始 query string，无 `?`；无查询参数时为**空字符串**
`Body的SHA256十六进制`	Body 原始字节（UTF-8）的 SHA-256，输出 **64** 位小写十六进制；无 Body 时对**空字节序列**做 SHA-256

Body 与 JSON：序列化应稳定（键顺序、无多余空格），建议与客户侧实际发送字节一致；常用为 `json.dumps(obj, separators=(',', ':'), ensure_ascii=False)` 再 UTF-8 编码后取哈希（与贵司网关约定一致即可）。

空 Body 的 SHA-256（十六进制）：

`e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855`

---

6. 多语言签名与请求示例

以下示例中 `KEY_ID` 、 `PRIVATE_KEY_BASE64` 均来自**客户本地生成**后的配置（环境变量或密钥管理系统），**勿**写入版本库。

6.1 Python

依赖： `pip install requests pynacl`

```python
import base64
import hashlib
import json
import os
import secrets
import time

import requests
from nacl.signing import SigningKey

KEY_ID = os.environ["ALTA_KEY_ID"]
PRIVATE_KEY_BASE64 = os.environ["ALTA_PRIVATE_KEY"]
BASE_URL = "https://api.altalex.com/v1"


def alta_request(method, path, params=None, json_data=None):
    timestamp = str(int(time.time()))
    nonce = secrets.token_hex(8)  # 16 个 hex 字符
    query = "&".join(f"{k}={v}" for k, v in sorted((params or {}).items()))
    body_str = "" if json_data is None else json.dumps(json_data, separators=(",", ":"), ensure_ascii=False)
    body_hash = hashlib.sha256(body_str.encode("utf-8")).hexdigest()
    message = f"{timestamp}|{nonce}|{method}|{path}|{query}|{body_hash}"

    private_key = SigningKey(base64.b64decode(PRIVATE_KEY_BASE64))
    signed = private_key.sign(message.encode("utf-8"))
    # PyNaCl 的 SignedMessage 前 64 字节为 detached 签名（亦可用 signed.signature，视版本而定）
    sig_b64 = base64.b64encode(bytes(signed)[:64]).decode("ascii")

    headers = {
        "X-API-Key-ID": KEY_ID,
        "X-Request-Timestamp": timestamp,
        "X-Request-Nonce": nonce,
        "X-Ed25519-Signature": sig_b64,
        "Content-Type": "application/json",
    }
    url = f"{BASE_URL}{path}"
    if query:
        url = f"{url}?{query}"
    return requests.request(method, url, headers=headers, json=json_data)


# 使用示例
# resp = alta_request("GET", "/contracts/list", params={"status": "active"})
```

```
# print(resp.json())
```

说明：PyNaCl 的 `sign()` 返回 `SignedMessage`，其字节形式前 **64** 字节为 detached 签名，对应 Header 中的 Base64；若贵司使用其他 API，请与所用库文档对齐。

6.2 Java

依赖（Maven 示例）： `org.bouncycastle:bcprov-jdk15on`

```java
import com.fasterxml.jackson.databind.ObjectMapper;
import org.bouncycastle.crypto.params.Ed25519PrivateKeyParameters;
import org.bouncycastle.crypto.signers.Ed25519Signer;
import org.bouncycastle.util.encoders.Base64;
import org.bouncycastle.util.encoders.Hex;

import java.net.HttpURLConnection;
import java.net.URL;
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.time.Instant;
import java.util.Map;
import java.util.UUID;
import java.util.stream.Collectors;

public class AltaClient {
    private static final String KEY_ID = System.getenv("ALTA_KEY_ID");
    private static final String PRIVATE_KEY_BASE64 = System.getenv("ALTA_PRIVATE_KEY");
    private static final String BASE_URL = "https://api.example.com/v1";

    public static void altaRequest(String method, String path, Map<String, String> params, Object body) throws Exception {
        String timestamp = String.valueOf(Instant.now().getEpochSecond());
        String nonce = UUID.randomUUID().toString().replace("-", "").substring(0, 16);
        String query = params == null || params.isEmpty()
                ? ""
                : params.entrySet().stream().map(e -> e.getKey() + "=" + e.getValue()).collect(Collectors.joining("&"));
        String bodyStr = body == null ? "" : new ObjectMapper().writeValueAsString(body);
        String bodyHash = sha256Hex(bodyStr);
        String message = String.join("|", timestamp, nonce, method, path, query, bodyHash);

        byte[] sk = Base64.decode(PRIVATE_KEY_BASE64);
        Ed25519PrivateKeyParameters privateKey = new Ed25519PrivateKeyParameters(sk, 0);
        Ed25519Signer signer = new Ed25519Signer();
        signer.init(true, privateKey);
        byte[] msgBytes = message.getBytes(StandardCharsets.UTF_8);
        signer.update(msgBytes, 0, msgBytes.length);
        byte[] sig = signer.generateSignature();
        String sigB64 = Base64.toBase64String(sig);


```

```java
        String urlStr = BASE_URL + path + (query.isEmpty() ? "" : "?" + query);
        HttpURLConnection conn = (HttpURLConnection) new URL(urlStr).openConnection();
        conn.setRequestMethod(method);
        conn.setRequestProperty("X-API-Key-ID", KEY_ID);
        conn.setRequestProperty("X-Request-Timestamp", timestamp);
        conn.setRequestProperty("X-Request-Nonce", nonce);
        conn.setRequestProperty("X-Ed25519-Signature", sigB64);
        conn.setRequestProperty("Content-Type", "application/json");
        // 写入 body、读取响应略
    }

    private static String sha256Hex(String input) throws Exception {
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] hash = digest.digest(input.getBytes(StandardCharsets.UTF_8));
        return Hex.toHexString(hash);
    }
}
```

6.3 Go

依赖：`golang.org/x/crypto/ed25519`、`github.com/google/uuid`

```go
package main

import (
    "crypto/ed25519"
    "crypto/sha256"
    "encoding/base64"
    "encoding/hex"
    "encoding/json"
    "fmt"
    "os"
    "sort"
    "time"

    "github.com/google/uuid"
)

const baseURL = "https://api.example.com/v1"

func altaRequest(method, path string, params map[string]string, body any) error {
    keyID := os.Getenv("ALTA_KEY_ID")
    skB64 := os.Getenv("ALTA_PRIVATE_KEY")
    timestamp := fmt.Sprintf("%d", time.Now().Unix())
    nonce := uuid.New().String()[:16]

    query := ""
    sep := ""
    keys := make([]string, 0, len(params))
    for k := range params {
        keys = append(keys, k)
    }
    sort.Strings(keys)
    for _, k := range keys {
        query += sep + k + "=" + params[k]
        sep = "&"
    }

    bodyBytes := []byte{}
    if body != nil {
        var err error
        bodyBytes, err = json.Marshal(body)
        if err != nil {
            return err
        }
    }
    sum := sha256.Sum256(bodyBytes)
    bodyHash := hex.EncodeToString(sum[:])
```

```go
    message := fmt.Sprintf("%s|%s|%s|%s|%s|%s", timestamp, nonce, method, path, query, bodyHash)

    sk, err := base64.StdEncoding.DecodeString(skB64)
    if err != nil {
        return err
    }
    sig := ed25519.Sign(sk, []byte(message))
    sigB64 := base64.StdEncoding.EncodeToString(sig)

    fmt.Printf("URL: %s%s\n", baseURL, path)
    fmt.Printf("X-API-Key-ID: %s\nX-Request-Timestamp: %s\nX-Request-Nonce: %s\nX-Ed25519-Signature: %s\n",
        keyID, timestamp, nonce, sigB64)
    return nil
}
```

6.4 Node.js

依赖：`npm install axios tweetnacl`

```javascript
const crypto = require('crypto');
const axios = require('axios');
const nacl = require('tweetnacl');

const KEY_ID = process.env.ALTA_KEY_ID;
const PRIVATE_KEY_BASE64 = process.env.ALTA_PRIVATE_KEY;
const BASE_URL = 'https://api.altalex.com/v1';

async function altaRequest(method, path, params = {}, body = null) {
  const timestamp = Math.floor(Date.now() / 1000).toString();
  const nonce = crypto.randomBytes(8).toString('hex');
  const query = new URLSearchParams(params).toString();
  const bodyStr = body ? JSON.stringify(body) : '';
  const bodyHash = crypto.createHash('sha256').update(bodyStr).digest('hex');
  const message = `${timestamp}|${nonce}|${method}|${path}|${query}|${bodyHash}`;

  const secretKey = Buffer.from(PRIVATE_KEY_BASE64, 'base64');
  const sig = nacl.sign.detached(Buffer.from(message, 'utf8'), secretKey);
  const sigB64 = Buffer.from(sig).toString('base64');

  const headers = {
    'X-API-Key-ID': KEY_ID,
    'X-Request-Timestamp': timestamp,
    'X-Request-Nonce': nonce,
    'X-Ed25519-Signature': sigB64,
    'Content-Type': 'application/json',
  };
  const url = `${BASE_URL}${path}${query ? `?${query}` : ''}`;
  return axios({ method, url, headers, data: body });
}
```

---

7. cURL（联调）

先用任一语言按第 6 节生成四个鉴权 Header，再填入：

```
curl -sS -X GET "https://example.com/v1/contracts/list?status=active" \
  -H "X-API-Key-ID: pk_live_..." \
  -H "X-Request-Timestamp: 1713345600" \
  -H "X-Request-Nonce: 8f4e2d9c1b5a3e7f" \
  -H "X-Ed25519-Signature: <Base64>"
```

---

8. 调试清单（签名失败）

若 HTTP **403** 或业务码提示签名失败，请逐项核对：

1. **私钥字节**：各语言库对 Ed25519「私钥」可能是 32 字节 seed 或 64 字节 expanded，须与生成时导出格式一致；Base64 解码后长度须符合该库要求。
2. **签名原文**：六段严格按第 5 节顺序，用 `|` 连接，段内无多余空格。
3. **HTTP 方法**：大写 `GET` / `POST` 等。
4. **路径**：与网关收到的 path 一致（注意是否含 `/v1` 前缀以贵司路由为准——**须与双方约定一致**；本规范示例为 path **不含** host，且与文档中 `BASE_URL` 拼接方式一致）。
5. **查询串**：与 URL 中实际发送的 query 一致；建议对参数名排序后拼接，避免与网关解析不一致。
6. **Body 哈希**：与线上发出的 body 字节完全一致（含 JSON 空格与 Unicode）；空 body 使用第 5 节给出的空串 SHA-256。

---

9. 密钥轮换与安全

- **轮换周期**：建议约**三年**评估更换；若怀疑泄露，应立即更换密钥对，并按**附录 B.1** 相同渠道与邮件结构向 AltaLex 提交新公钥（主题中建议注明「密钥轮换」）；是否沿用原 Key ID 或签发新 Key ID 以双方约定为准。
- **并行窗口**：新旧公钥短期并存等策略以 AltaLex 书面约定为准。
- **传输**：私钥不得进入邮件、聊天、工单；公钥注册建议使用**附录 A** 中的加密与分通道方式。

附录 A 公钥与配置登记渠道

A.1 当前渠道（公钥登记）

项目	说明
主渠道	**电子邮件**（将 `key_id` 、公钥 Base64、组织信息发给 AltaLex 指定接口邮箱；**勿在邮件中包含私钥**）
收件地址	由商务/合同或开通通知中提供的 **AltaLex API 密钥登记邮箱** 为准（本文不写死地址，避免变更不同步）
安全建议	使用 **PGP 加密邮件**或**加密附件**；若用附件密码，密码须通过 **另一独立通道**（如钉钉/电话）告知 AltaLex，勿与密文同信道
后续扩展	若 AltaLex 提供**控制台上传公钥**等渠道，以届时发布的规范版本为准

A.2 其他配置（IP 白名单等）

可选 **IP 白名单**的申请与回函同样采用**电子邮件**；往来模板见**附录 C**。

附录 B 邮件模板（公钥注册与接入确认）

以下模板中 `{占位符}` 由发件方替换；示例组织名可删改。

B.1 客户 → AltaLex：公钥注册申请

```
主题：[AltaLex] API 公钥注册 — {客户组织简称}

━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
公钥注册信息（请勿在邮件中包含私钥）
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

组织 / 账号标识：{CLIENT_ORG_ID}
联系人邮箱：{CONTACT_EMAIL}
期望绑定的 UID / 角色（如已知）：{UID} / {ROLE}

Key ID（由贵司生成，建议 pk_live_ 前缀 + 高熵随机串；勿与 AltaLex 已登记的 Key ID 主键冲突，否则登记将被退回）：
{KEY_ID}

Ed25519 公钥（Base64，verify key 编码后的一行）：
{PUBLIC_KEY_B64}

可选：如贵司有固定出口 IP，可一并列出以便 AltaLex 启用 IP 白名单（详见附录 C）。

说明：
— 密钥对在贵司服务器本地生成；具体步骤见本规范第 2、3 节。私钥切勿通过邮件、IM 或工单发送给 AltaLex。
— 若怀疑私钥泄露，请立即生成新的 Key ID 与密钥对，并重新发送本模板申请轮换（主题建议注明「密钥轮换」）。
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
```

B.2 AltaLex → 客户：接入确认（不含私钥）

```
主题：[AltaLex] API 接入确认 — {客户组织简称}

━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
登记回执（不含私钥）
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

Key ID（与贵司申请一致，已登记）：{KEY_ID}
登记有效期：{DATE_FROM} 至 {DATE_TO}（建议满三年前安排轮换；泄露风险下可随时按附录 B.1 申请更新公钥）

随本确认一并提供（或此前已提供）：
— 《AltaLex API接入技术规范》版本 {SPEC_VERSION} / 修订日期 {SPEC_DATE}（含 Endpoint、鉴权 Header、签名规则、密钥生成步骤与多语言示例；**邮件正文不重复技术细节**）

接入步骤请贵司技术同事**仅以该规范正文为唯一技术依据**完成集成与调试。

安全与运维提示：
1. Ed25519 请求签名；私钥仅在贵司环境配置，从未由 AltaLex 传输或保存（详见规范正文）。
2. 可选 IP 白名单：如有固定出口 IP，请回复此邮件提供网段列表，AltaLex 将启用绑定（附录 C）。
3. 建议轮换前与 AltaLex 确认并行验签窗口（若有）。

━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
```

附录 C 可选 IP 白名单：邮件往来模板

C.1 客户 → AltaLex：申请启用绑定

```
主题：[AltaLex] API IP 白名单申请 — {客户组织简称}

我司已登记 Key ID：{KEY_ID}
请为上述 Key ID 启用 IP 白名单，允许以下出口网段访问：
{CIDR_1}, {CIDR_2}, ...

联系人：{NAME} / {PHONE_OR_EMAIL}
```

C.2 AltaLex → 客户：已启用回函

```
主题：Re: [AltaLex] API IP 白名单申请 – {客户组织简称}

已为 Key ID {KEY_ID} 启用 IP 白名单，仅允许以下网段访问：
{CIDR_1}, {CIDR_2}, ...

如有出口变更，请发邮件更新列表。
```

---

10. 修订记录

版本	日期	说明
1.0.0	2026-04-23	首版对外技术规范(首期测试对接版本后续如发生变更会及时同步)